燕山大学

编译原理课程设计报告

题目 递归下降 LL(1)语法分析

学　　院　信息科学与工程学院

年级专业　2023 级计算机 5 班

学生姓名　李济岑

学生学号　202211090252

设计日期　2026 年 1 月 4 日-1 月 8 日

摘要

本课程设计以“递归下降 LL(1) 表达式语法分析”为核心内容，目标是将传统命令行式语法分析器升级为更适合教学演示与作业验收的可视化桌面工具。项目采用 Python + PyQt5 完成 GUI 交互层开发，并在核心逻辑中实现了 词法分析（Tokenize）→ 递归下降预测分析（LL(1)）→ 最左推导记录 → 语法树构建与渲染 的完整链路。系统支持实时输入解析：用户在输入框中键入表达式后，程序会自动节流触发分析，实时输出 Tokens 列表、合法性判定、最左推导序列以及 ASCII 语法树；同时提供批量处理入口，将多条表达式解析结果统一写出为文本报告，方便验收与回归测试。

用户在 GUI 输入框中键入表达式后，系统通过节流机制自动触发分析过程，实时输出 Tokens 序列、合法性判定信息、最左推导过程以及 ASCII 形式的语法树展示；同时提供批量处理入口，可读取 input.txt 中的多条表达式并生成结构化 output.txt 报告，满足课堂验收对“批处理 + 可追溯输出”的常见要求。该工具将抽象的 LL(1) 分析流程具象化，提升学习体验，并强化对编译原理自顶向下分析思想的理解与工程实现能力。

关键词：递归下降；LL(1)语法分析；词法分析；语法分析；Python；PyQt。

目录

摘要 I

目录 II

一、概述 1

1. 目的与意义 1

2. 项目环境 1

3. 目标任务 1

4. 课程设计计划 2

二、基本原理 2

1. 递归下降 2

2. LL(1)语法分析 3

2. 代码设计原理 4

三、总体设计 5

1. 技术路线 5

2. 项目结构 5

3. 功能模块 5

4. 项目流程 6

四、编码实现 8

1. 开发环境 8

2. 主要代码 9

五、测试运行 14

1. 界面展示 14

2. 单点测试 16

3. 批量测试 18

六、总结 23

七、心得体会 23

附录 24

一、概述

1. 目的与意义

在编译原理学习中，“自顶向下分析”“预测分析表”“最左推导”“语法树”等概念往往较抽象，学生容易停留在公式与符号层面，难以形成稳定的过程性理解。本项目将 LL(1) 语法分析过程与课堂常见表达式文法结合，通过可视化方式呈现完整推导链路，具有以下意义：

1. 教学可解释性增强：

将“看不见的分析过程”转化为“可观察的 Tokens / 推导 / 语法树”，让学生能够把每一次产生式展开与输入符号的匹配过程对应起来。

2. 工程化能力训练：

不仅实现语法分析算法，还完成 GUI 交互、异常提示、批量处理、格式化输出等工程功能，符合课程设计对综合能力的要求。

3. 便于验收与自测：

批量处理机制使得多组样例可以一键解析并输出报告，降低人工检查成本，也便于定位典型错误用例。

2. 项目环境

- 语言/解释器：Python 3.10+
- 主要依赖：PyQt5
- 运行方式：
  - GUI 模式：执行 python3 main.py 启动桌面程序，输入框内容变化后自动触发分析
  - 批量模式：点击“批量处理”按钮选择 input.txt，系统逐行处理并生成 output.txt

3. 目标任务

1. 表达式 LL(1) 语法分析的完整实现

针对表达式文法 E, E', T, T', F 设计递归下降流程，正确处理 + - * / 的优先级与结合性，并在解析过程中构建语法树结构。

2. 错误定位精确且信息充分

词法阶段：定位非法字符并报告位置

语法阶段：当 token 不匹配期望集合时，错误对象携带“位置 + 实际 token + 期望 token 集合”，便于学生快速定位并理解错误原因。

3. 可视化交互体验优化

采用卡片式布局与分栏展示，信息层次清晰；通过定时器节流（例如 250ms）避免每次按键都触发解析造成闪烁与性能浪费，使“实时分析”既灵敏又稳定。

4. 批量分析与验收输出

提供批量处理入口，将多条表达式的结果统一生成报告文件，输出内容包括 Tokens、推导序列、语法树或错误信息，方便助教检查与对比。

4. 课程设计计划

在本课程设计中，按照以下顺序进行程序的开发：

1. 需求分析阶段

明确文法范围（仅表达式）、输入输出格式、GUI 展示组件（Tokens / 推导 / 树 / 状态提示）以及验收需要的批处理能力。

2. 核心算法阶段

先实现词法分析器 tokenize，再实现递归下降 Parser，并同步设计“最左推导记录机制”，保证推导与语法树能同时产出；最后设计异常类统一错误表达。

3. 界面与交互阶段

搭建主窗体、输入区、输出区与状态栏，确定布局策略（如 QSplitter）；引入 QTimer 做输入节流，提高用户体验。

4. 批量输出阶段

设计 _process_expression 的通用接口，使 GUI 实时模式与批量模式共用同一套词法/语法核心；实现文件读取、空行处理、输出组织与错误兜底。

5. 测试与文档阶段

准备典型合法/非法样例，覆盖运算符优先级、括号嵌套、缺失符号、非法字符等情况；补充 README 与课程报告，形成可复现的交付材料。

二、基本原理

1. 递归下降

递归下降（Recursive Descent Parsing）是一类典型的自顶向下语法分析方法，其核心思想是：为每个非终结符编写一个同名的递归函数，函数内部按照产生式展开并逐步消费输入 token。它具有以下特点：

过程直观：每次调用 parse_X() 等价于“尝试用非终结符 X 推导出当前剩余输入”，与教材推导过程一致。

依赖文法性质：若文法存在左递归（如 E -> E + T），递归下降会出现无限递归，因此通常需要先做文法改写或消除左递归。

匹配与回溯策略：在 LL(1) 场景中，由于对每一步选择都具备确定性，不需要回溯；程序只需检查一个 lookahead token 即可决定采用哪条产生式。

本项目中，递归下降的实现体现在 Parser 类中：parse_E、parse_Ep、parse_T、parse_Tp、parse_F 六个函数与文法一一对应，解析过程通过 consume() 严格匹配终结符，保证“读入即验证”。

2. LL(1)语法分析

LL(1) 的含义可以拆解为：

- 第一个 L：从左到右扫描输入（Left-to-right）。
- 第二个 L：构造最左推导（Leftmost derivation）。
- (1)：使用 1 个向前看符号（1-token lookahead）做预测。

本项目采用的表达式文法如下（已消除左递归并进行因子化处理），这是经典的 LL(1) 表达式文法：

```
E  -> T E'
E' -> + T E' | - T E' | ε
T  -> F T'
T' -> * F T' | / F T' | ε
F  -> ( E ) | id | num
```

该文法的关键点在于：

优先级由层级保证：E 负责加减，T 负责乘除，F 负责基本因子（括号/标识符/数字）。因为 E 必须先解析出 T，而 T 在内部又必须先解析 F，所以乘除天然比加减“更深一层”，从而具有更高优先级。

ε 产生式依赖 FOLLOW 集判断：例如 E' 何时取 ε？当后续符号属于 FOLLOW(E')（也就是 FOLLOW(E)）时可以收束。对于本表达式文法，有：

FOLLOW(E) = { ')', EOF }，所以当 lookahead 为) 或文件结束时，E' 归约为 ε。

FOLLOW(T) = { '+', '-', ')', EOF }，所以当 lookahead 为 + -) EOF 时，T' 归约为 ε。

在代码中，这种“预测分析表”的效果并没有显式用二维表实现，而是以 if/elif 分支隐式体现：例如 parse_Ep() 看当前 token 是 PLUS/MINUS 还是 RPAREN/EOF 来选择产生式，这与 LL(1) 的预测选择完全等价。

2. 代码设计原理

为了让“算法正确性”和“教学可读性”同时成立，项目在代码层面遵循了以下设计原则：

1. 数据结构清晰可解释：

使用 @dataclass 定义 Token(type, value, position) 与 Node(label, value, children)，让词法单元和语法树节点具备清晰字段，便于调试与展示。

2. 职责分离：

词法分析（tokenize）、语法分析（Parser）、格式化输出（format_tokens/format_derivation/format_tree）、GUI（ParserWindow）逻辑相对独立，既便于阅读，也便于后续扩展（例如增加 FIRST/FOLLOW 展示或导出图形树）。

3. 统一错误模型：

用 LexerError 表示词法阶段无法识别字符，用 ParserError 表示语法阶段不匹配，并携带 position / found / expected 等上下文，保证 GUI 与批量模式能共享同一套错误信息。

4. 面向教学的“过程记录”机制：

通过 record(lhs, rhs) 维护 current_sentential_form，在每次展开产生式时记录最左推导步骤，使得“推导序列”与“递归下降调用链”同步增长，利于课堂讲解。

三、总体设计

1. 技术路线

本系统使用了 Python 的面向对象与数据类建模技术，实现了 Token、语法树节点、异常信息等核心数据结构的规范表达功能；使用了手写词法分析的逐字符扫描技术，实现了对数字、标识符、运算符与括号等词法单元的自动识别与定位功能；使用了 LL(1) 递归下降预测分析的语法分析技术，实现了对表达式合法性判定、最左推导生成与语法树构建的功能；使用了 ASCII 树文本渲染的格式化输出技术，实现了将内部语法树结构转化为可在 GUI/文件中直接阅读的树形文本的功能；

使用了 PyQt5 桌面 GUI 框架的组件化界面开发技术，实现了输入实时触发、结果分区展示、提示信息反馈与批量处理交互的功能；使用了 QTimer 输入节流的事件驱动技术，实现了“输入稳定后再解析”的实时体验优化功能，从而避免频繁解析造成的卡顿与界面闪烁。

2. 项目结构

项目采用“轻量级单文件承载核心逻辑 + 资源文件辅助展示”的结构，便于课程验收与阅读；同时在单文件内部按“异常与数据结构 → 词法 → 语法 → 格式化 → GUI → 批量处理”顺序组织，使逻辑层次清晰。

```
.
├── ./input.txt          // 用于批量测试的输入语句文件
├── ./logo.ico           // GUI 设计的 icon 图标
├── ./main.py            // 主要代码文件
├── ./output.txt         // 批量测试的输出文件
└── ./README.md          // 说明文件

1 directory, 5 files
```

3. 功能模块

本课程设计的功能模块图如下图所示。

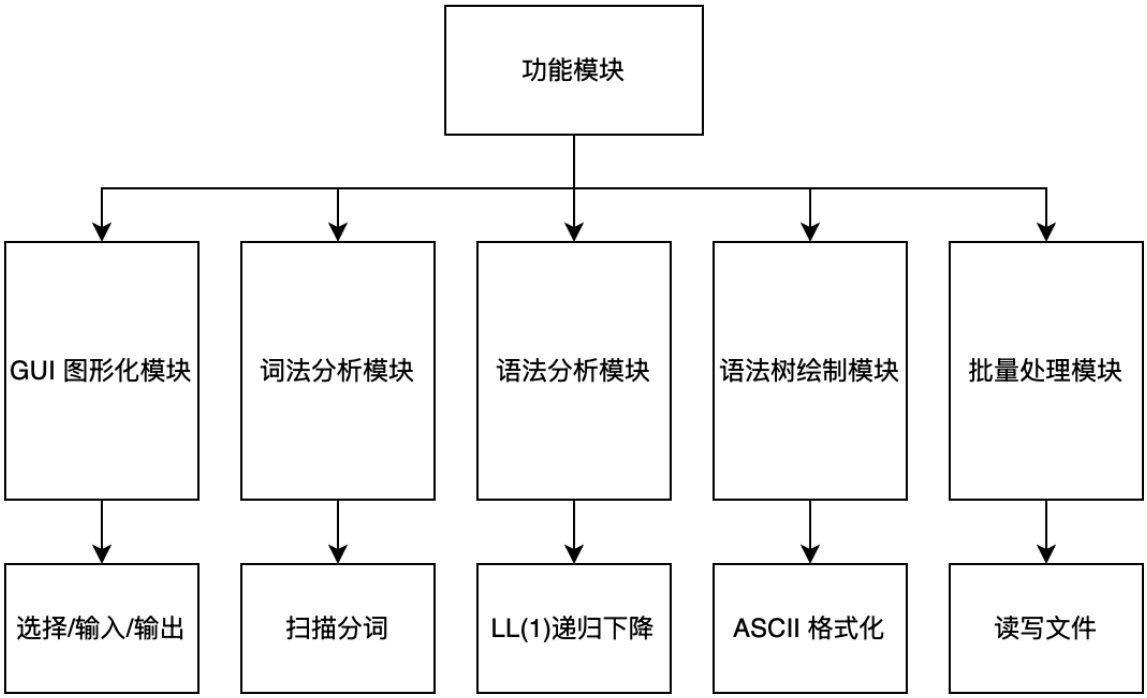


图 0-1 功能模块图

4. 项目流程

系统运行时先启动 GUI 主窗口，用户在输入框内键入表达式后，界面不会在每一次按键立刻解析，而是通过定时器机制等待短暂输入停顿；

当触发解析后，程序首先对表达式进行词法分析，生成带位置的 Token 序列并追加 EOF 作为结束标记；

随后将 Token 序列交给递归下降语法分析器，语法分析器以 lookahead token 为依据选择产生式展开，在消费 token 的同时构建语法树节点，并把每一次产生式替换记录为“最左推导”步骤；

若词法阶段出现非法字符，系统直接给出词法错误提示并停止后续解析；若语法阶段出现不匹配，系统抛出带位置与期望集合的语法错误，界面在保留 Tokens 与已生成推导步骤的同时提示“本次失败未生成语法树”，帮助定位问题；

当解析成功时，Tokens、推导序列与语法树将分别被格式化后显示在三个结果面板中，同时状态区显示“合法表达式”。在批量处理模式下，系统从文本文件逐行读取表达式（跳过空行），对每一条表达式复用同一条分析管线，并将每个用例的结果以统一模板写入 output.txt，最终在提示区与弹窗中给出完成信息。

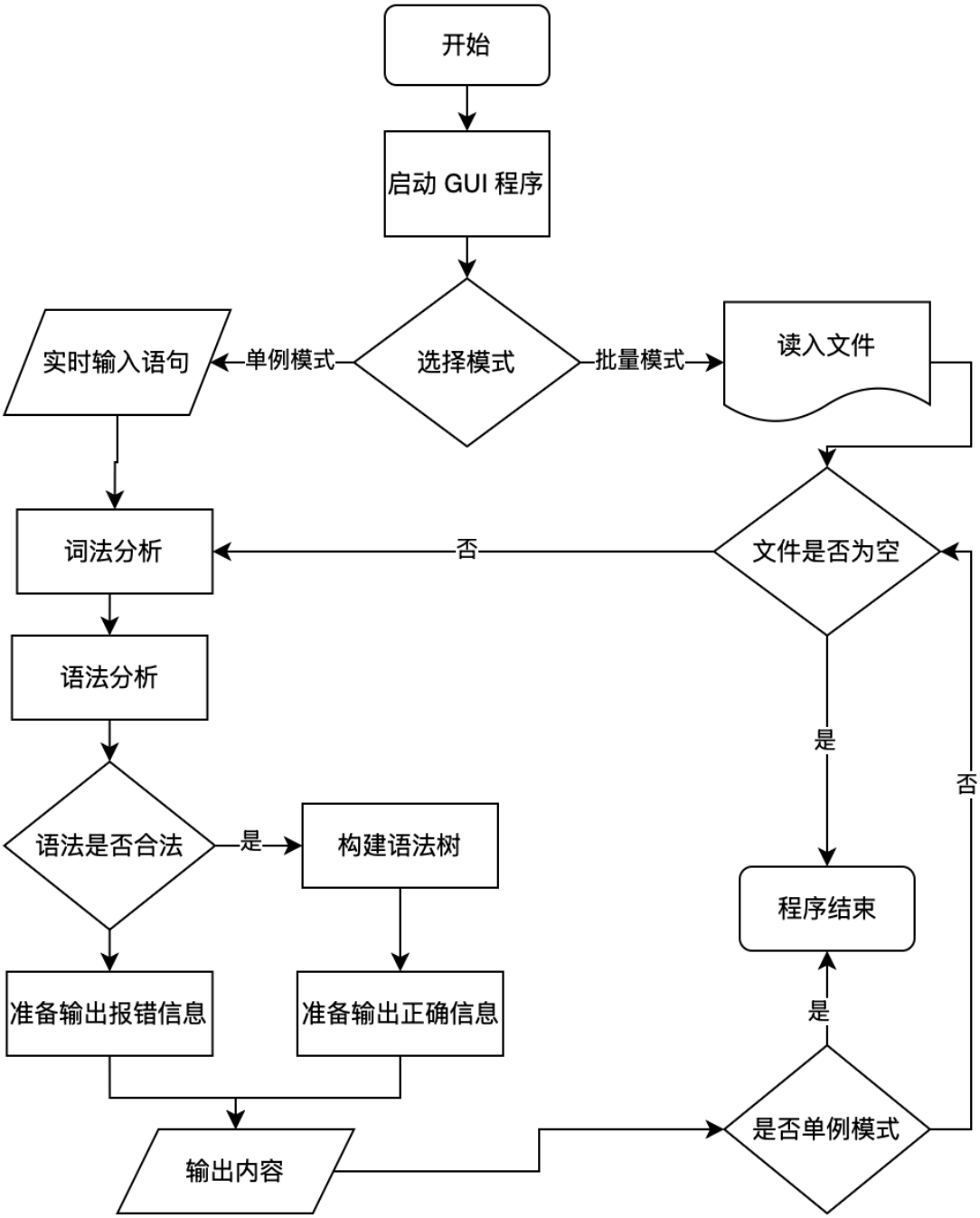

开始
启动 GUI 程序
选择模式
单例模式
批量模式
实时输入语句
读入文件
词法分析
否
文件是否为空
语法分析
是
否
语法是否合法
是
构建语法树
程序结束
准备输出报错信息
准备输出正确信息
是
是否单例模式
输出内容

图 0-2 程序流程图

四、编码实现

1. 开发环境

本项目在 Windows / macOS / Linux 上均可运行，关键在于正确安装 Python 与 PyQt5，并确保以正确方式启动脚本。

1）Python 安装与版本确认

推荐安装 Python 3.10 或以上版本。

验证版本：

```
python --version
Python 3.10.0
```

2）创建虚拟环境

使用虚拟环境可以避免全局包冲突：

```
conda create -n Script python=3.10
conda activate Script
```

3）安装依赖

本项目 GUI 依赖 PyQt5，安装命令如下：

```
pip install PyQt5
```

4）运行方式

启动后，在输入框输入表达式即可实时解析；点击“批量处理”选择文本文件可输出 output.txt。

```
python3 main.py
```

5）常见运行配置建议

若高 DPI 下界面缩放不合适，可设置 Qt 缩放（可选）：

macOS/Linux：export QT_AUTO_SCREEN_SCALE_FACTOR=1

Windows：在系统显示设置或 Qt 环境变量中调整。

2. 主要代码

注：下列仅展示核心功能的关键实现片段与设计思路；完整代码见报告末尾附录。

2.1 词法分析：将字符流切分成 Token 流

核心目标是：识别数字 NUM、标识符 ID、运算符与括号，并记录位置 position 以支持错误定位（PLUS/MINUS/MUL/DIV/LPAREN/RPAREN）。

核心代码如下：

```
def tokenize(text: str) -> List[Token]:
    tokens: List[Token] = []
    i = 0
    length = len(text)
    while i < length:
        ch = text[i]
        if ch.isspace():
            i += 1
            continue
        if ch.isdigit():
            start = i
            while i < length and text[i].isdigit():
                i += 1
            tokens.append(Token("NUM", text[start:i], start))
            continue
        if ch.isalpha() or ch == "_":
            start = i
            i += 1
            while i < length and (text[i].isalnum() or text[i] == "_"):
                i += 1
            tokens.append(Token("ID", text[start:i], start))
            continue
        single_char_tokens = {...}
        if ch in single_char_tokens:
```

```
            tokens.append(Token(single_char_tokens[ch], ch, i))
            i += 1
            continue
        raise LexerError(f"无法识别的字符 '{ch}' (位置 {i})")
    tokens.append(Token("EOF", "", length))
    return tokens
```

实现要点：

用 while 扫描字符串，遇到数字/标识符走“吞噬式”读取，保证 123 作为一个整体 token。

单字符运算符用字典映射，结构简单且便于扩展。

对非法字符立即抛 LexerError，并带出位置索引，GUI 能直接显示定位信息。

追加 EOF 作为输入结束标记，便于语法分析器收束并检测“多余符号”。

2.2 语法分析：LL(1)递归下降与错误模型

consume() 是语法分析中最关键的“断言”函数：如果当前 token 类型不符合预期，就抛 ParserError，并携带 found/expected/position。

核心代码如下：

```
def consume(self, token_type: str, expected_desc: Optional[str] = None) -> Token:
    token = self.current()
    if token.type == token_type:
        return self.advance()
    expected = expected_desc or token_type
    raise ParserError(
        message=f"位置 {token.position} 处需要 {expected}",
        position=token.position,
        found=token.type,
        expected=[expected],
    )
```

递归下降函数与产生式一一对应，以 E' 为例，它同时体现了“选择产生式”与“ε 归约”逻辑：

```
def parse_Ep(self) -> Node:
    token = self.current()
    if token.type == "PLUS":
        self.record("E'", ["+", "T", "E'"])
        op = self.consume("PLUS")
        t_node = self.parse_T()
        ep_node = self.parse_Ep()
        return Node("E'", children=[Node("op", value=op.value), t_node, ep_node])
    if token.type == "MINUS":
        self.record("E'", ["-", "T", "E'"])
        op = self.consume("MINUS")
        t_node = self.parse_T()
        ep_node = self.parse_Ep()
        return Node("E'", children=[Node("op", value=op.value), t_node, ep_node])
    if token.type in {"RPAREN", "EOF"}:
        self.record("E'", [])
        return Node("E'", value=" ε ")
    raise ParserError(...)
```

实现要点：

PLUS/MINUS 对应两条显式产生式，递归调用形成“链式”表达结构。

RPAREN/EOF 对应 FOLLOW(E) 的符号集合，代表 E' 可以收束为 ε。

不属于以上集合则抛出“无法归约”的错误，并给出 expected 集合，帮助定位。

2.3 最左推导记录：解析过程转换为可读推导

需要维护一个“当前句型”，每次展开产生式时替换最左侧的对应非终结符：

核心代码如下：

```
def record(self, lhs: str, rhs: List[str]):
    try:
        idx = next(i for i, sym in enumerate(self.current_sentential_form) if sym == lhs)
    except StopIteration:
```

```
        return
    if rhs:
        self.current_sentential_form = (
            self.current_sentential_form[:idx] + rhs + self.current_sentential_form[idx + 1 :]
        )
    else:
        self.current_sentential_form = (
            self.current_sentential_form[:idx] + self.current_sentential_form[idx + 1 :]
        )
    step = " ".join(self.current_sentential_form) if self.current_sentential_form else "ε"
    self.derivation_steps.append(step)
```

实现要点：

current_sentential_form 用列表存储句型符号序列，便于替换与拼接。

record() 找到最左出现的 lhs，替换为 rhs（空列表代表 ε）。

推导序列 derivation_steps 用于 GUI“最左推导”面板展示，能直接看到从 E 到终结符串的演化过程。

2.4 语法树渲染：打印字符树结构

为了让语法树无需额外绘图库即可展示，项目采用文本树结构渲染：

核心代码如下：

```
def format_tree(node: Node) -> str:
    lines: List[str] = []
    def walk(current: Node, prefix: str = "", is_last: bool = True, is_root: bool = True):
        label = current.label if current.value is None else f"{current.label}: {current.value}"
        if is_root:
            lines.append(label)
        else:
            connector = " └─── " if is_last else " ├─── "
            lines.append(prefix + connector + label)
        new_prefix = prefix + ("      " if is_last else " │    ")
```

```
        for idx, child in enumerate(current.children):
            walk(child, new_prefix, idx == len(current.children) - 1, False)
    walk(node)
    return "\n".join(lines)
```

实现要点：

深度优先遍历语法树，使用 `└──/├──/│` 表示层级关系，输出直观。

prefix 与 is_last 共同决定连接符与缩进，使树形结构对齐。

2.5 批量处理：统一 复用和报告写出

批量处理通过 _process_expression() 复用单条表达式的解析逻辑，保证实时模式与批量模式的一致性，再将结果写入 output.txt：

核心代码如下：

```
def _process_expression(self, expr: str):
    ...
    tokens = tokenize(expr)
    parser = Parser(tokens)
    tree = parser.parse()
    info["tokens"] = format_tokens(tokens)
    info["derivation"] = format_derivation(parser.derivation_steps)
    info["tree"] = format_tree(tree)
    return info
```

2.6 遇到的困难和解决

Q1：递归下降出现无限递归，难点在于怎么避免：

A： 直接用表达式原始文法（如 E -> E + T | T）会存在左递归，递归下降在调用 parse_E() 时会不断再次调用 parse_E()，导致无限递归。解决方法是将文法改写为无左递归形式，引入 E'、T' 结构（E -> T E'、E' -> + T E' | ε 等），使递归沿着输入推进，从根本上消除无限递归风险。

Q2：E' 和 T' 的 ε 产生式什么时候选，如何保证 LL(1) 的确定性：

A： ε 的选择需要依赖 FIRST/FOLLOW 原理。实现中用 lookahead token 判断：当 lookahead 属于 FOLLOW 集时，选择 ε（例如 E' 在) 或 EOF 处归约；T' 在 + -) EOF 处归约）。这样每个非终结符在任意 lookahead 下都有唯一决策，从而保证 LL(1) 的确定性，避免“到底该展开还是该结束”的歧义。

Q3：如何在递归下降过程中正确记录“最左推导”：

A： 难点在于推导是对“句型”进行替换，而递归下降是对“函数调用栈”进行展开，两者并不天然等价。若只在函数调用时简单打印产生式，很容易出现“替换的不是最左非终结符”或“推导步骤顺序不一致”。项目通过维护 current_sentential_form（句型符号序列），在 record(lhs, rhs) 中定位最左 lhs 并替换为 rhs，保证推导序列严格符合最左推导定义，并与教材推导形式一致。

五、测试运行

1. 界面展示



5 - 1 GUI 界面图

主要界面布局如下：

1. 顶部标题区

主标题为“递归下降 LL(1)语法分析”，用于强调项目主题。副标题“实时递归下降推导 · Tokens / 判定 / 推导 · 语法树”概括了系统输出的四类核心信息，起到“功能目录”的作用。窗口最上方显示“燕山大学编译原理课程设计”，与课程属性一致。

2. 输入区

文本提示“输入表达式，系统将自动实时分析：”明确交互方式。中间长输入框用于输入表达式，包含占位示例“例如：a+3*(b-2)”，引导用户输入合法样例。右侧有两个按钮：

“批量处理”：打开文件选择对话框，选择 txt 后批量解析并生成 output.txt。

“清空”：清空输入框并重置 Tokens/推导/语法树/提示信息，便于开始新用例演示。

3. 状态判定区

显示“判定：等待输入”，用于给出解析总结果状态。下方提示“在输入框内键入表达式即可实时看到结果”，提示用户实时解析的使用方式。实际运行中该区域会随解析结果变化为“合法表达式/不合法表达式/词法错误/实时分析中”等，并通过颜色区分状态（中性/成功/失败）。

4. 提示信息区

标题“提示信息”，下方是一个只读文本区域。当解析失败时，此处会显示错误定位（索引位置）、遇到的 token 类型、期望集合以及修复建议；当解析成功时显示“解析成功，可结合推导与语法树讲解”等引导语。该区域在教学场景中非常关键：它把错误从“结果不对”变成“哪里不对、为什么不对、下一步怎么改”。

5. 结果展示区

左侧上下分割为两个面板：

- Tokens 面板：逐行列出 token 序号、类型、字面量与位置（pos），用于展示词法分析结果与定位依据。

- 最左推导面板：按步编号展示推导序列（1. 2. 3. …），用于对应教材推导过程。

右侧为语法树面板：

用 ASCII 树展示语法树结构，能直观看到表达式的结构层次与运算符优先级（例如乘除节点会更深）。

三个面板均为只读文本框，支持滚动查看；布局采用可拖动分割（QSplitter），在不同屏幕尺寸下可自适应调整显示比例。

2. 单点测试





图 5－2 单点测试图

单点测试样例输入：1+2*，预期为错误语法。

测试结果如图 5-2 上图，判定为不合法表达式，仅进行了词法分析和最左推导，未绘制语法树。

单点测试样例输入：1+2*3，预期为正确语法。

测试结果如图 5-2 下图，判定合法表达式，进行了词法分析、最左推导和绘制语法树。输出结果如下：

Tokens：

```
0: NUM='1' (pos 0)
1: PLUS='+' (pos 1)
2: NUM='2' (pos 2)
3: MUL='*' (pos 3)
4: NUM='3' (pos 4)
5: EOF (pos 5)
```

最左推导：

```
1. E
2. T E'
3. F T' E'
4. num T' E'
5. num E'
6. num + T E'
7. num + F T' E'
8. num + num T' E'
9. num + num * F T' E'
10. num + num * num T' E'
11. num + num * num E'
12. num + num * num
```

语法树：

```
E
├── T
│   ├── F
│   │   └── num: 1
│   └── T': ε
└── E'
├── op: +
├── T
│   ├── F
│   │   └── num: 2
│   └── T'
│   ├── op: *
│   ├── F
│   │   └── num: 3
│   └── T': ε
└── E': ε
```

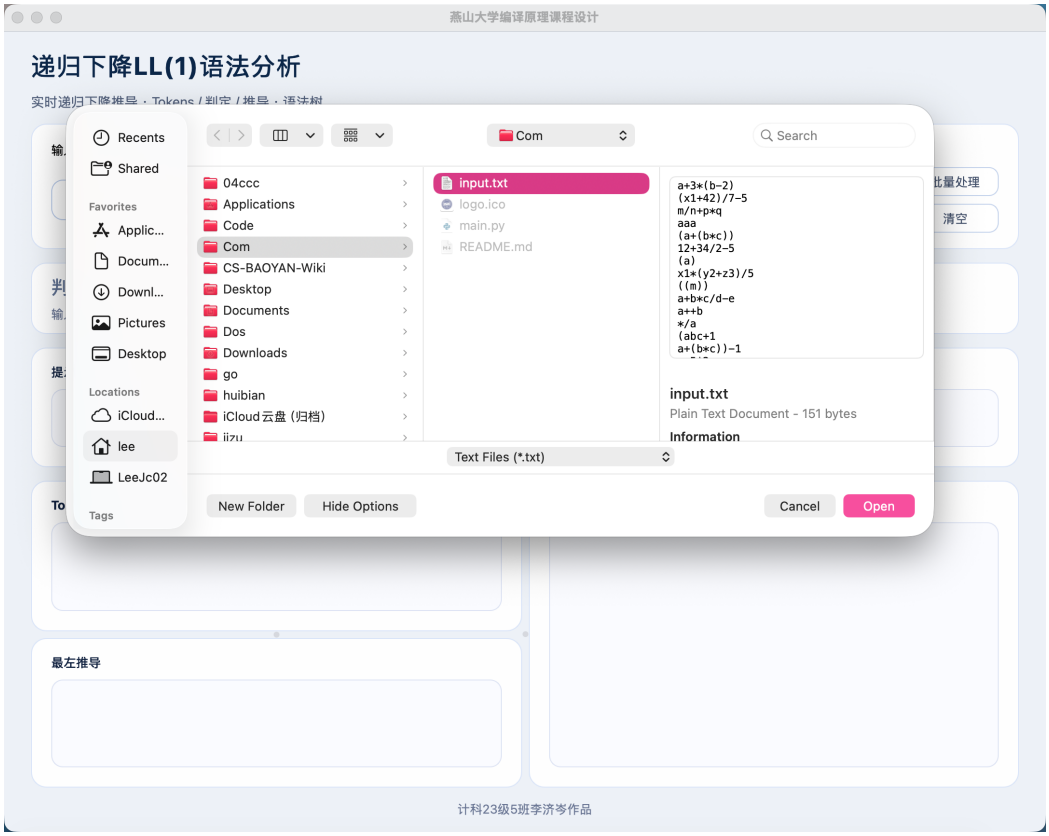
从单例测试的展示结果可以看出，系统能够在输入框中对表达式进行稳定的实时解析：词法分析会输出完整、带位置标注的 Tokens 序列；语法分析在 LL(1) 递归下降过程中同步生成最左推导步骤与语法树结构，并在界面三栏中清晰呈现。

当输入合法时，状态区能够准确显示“合法表达式”，语法树层次结构能够反映括号嵌套与运算符优先级；当输入不合法或包含非法字符时，系统能够区分词法错误与语法错误，并在提示信息区给出错误位置、遇到的符号与期望集合，使得错误原因可解释。

3. 批量测试

点击批量处理后，将进行输入文件的选择，输入文件要求是一行为一个测试点语句，文件以 txt 格式进行存储。选择文件后将对每一个测试点语句进行工作流的分析，然后以 output.txt 文件进行输出测试点内容，并在全部语句的分析工作结束后进行统计，并在 GUI 显示弹窗已结束。

批量测试结果如图 5-3：


燕山大学编译原理课程设计
递归下降LL(1)语法分析
实时递归下降推导 · Tokens / 判定 / 推导 · 语法树
Recents
Shared
Favorites
Applic...
Docum...
Downl...
Pictures
Desktop
Locations
iCloud...
lee
LeeJc02
Tags
Com
Search
04ccc
Applications
Code
Com
CS-BAOYAN-Wiki
Desktop
Documents
Dos
Downloads
go
huibian
iCloud 云盘 (归档)
input.txt
logo.ico
main.py
README.md
a+3*(b-2)
(x1+42)/7-5
m/n+p*q
aaa
(a+(b*c))
12+34/2-5
(a)
x1*(y2+z3)/5
((m))
a+b*c/d-e
a++b
*/a
(abc+1
a+(b*c))-1
input.txt
Plain Text Document - 151 bytes
Information
Text Files (*.txt)
New Folder
Hide Options
Cancel
Open
批量处理
清空
最左推导
计科23级5班李济岑作品


燕山大学编译原理课程设计
递归下降LL(1)语法分析
实时递归下降推导 · Tokens / 判定 / 推导 · 语法树
输入表达式，系统将自动实时分析：
例如：a+3*(b-2)
批量处理
清空
判定：批量完成
output.txt 已生成
提示信息
批量处理完成，共 20 条输入。
输出文件：/Users/lee/Com/output.txt
已批量完成
OK
Tokens
语法树
最左推导
计科23级5班李济岑作品

图 5－3 批量测试图

局部测试用例如下：（input.txt)

```
a+3*(b-2)
*/a
```

局部测试输出如下：（output.txt)

```
[用例 1] 原始：a+3*(b-2)
结果：合法
Tokens:
0: ID='a' (pos 0)
1: PLUS='+' (pos 1)
2: NUM='3' (pos 2)
3: MUL='*' (pos 3)
4: LPAREN='(' (pos 4)
5: ID='b' (pos 5)
6: MINUS='-' (pos 6)
7: NUM='2' (pos 7)
8: RPAREN=')' (pos 8)
9: EOF (pos 9)
最左推导:
1. E
2. T E'
3. F T' E'
4. id T' E'
5. id E'
6. id + T E'
7. id + F T' E'
8. id + num T' E'
9. id + num * F T' E'
10. id + num * ( E ) T' E'
11. id + num * ( T E' ) T' E'
12. id + num * ( F T' E' ) T' E'
```

13. id + num * (id T' E') T' E'

14. id + num * (id E') T' E'

15. id + num * (id - T E') T' E'

16. id + num * (id - F T' E') T' E'

17. id + num * (id - num T' E') T' E'

18. id + num * (id - num E') T' E'

19. id + num * (id - num) T' E'

20. id + num * (id - num) E'

21. id + num * (id - num)

语法树:

```
E
├── T
│   ├── F
│   │   └── id: a
│   └── T': ε
└── E'
    ├── op: +
    ├── T
    │   ├── F
    │   │   └── num: 3
    │   └── T'
    │       ├── op: *
    │       ├── F
    │       │   ├── (: (
    │       │   ├── E
    │       │   │   ├── T
    │       │   │   │   ├── F
    │       │   │   │   │   └── id: b
    │       │   │   │   └── T': ε
    │       │   │   └── E'
    │       │   │       ├── op: -
```

```
        │       │   │       ├── T
        │       │   │       │   ├── F
        │       │   │       │   │   └── num: 2
        │       │   │       │   └── T': ε
        │       │   │       └── E': ε
        │       │   └── ): )
        │       └── T': ε
        └── E': ε
```

[用例 2] 原始：*/a

结果：不合法

语法错误：索引 0，遇到 MUL，期望 (, id, num

Tokens:

0: MUL='*' (pos 0)

1: DIV='/' (pos 1)

2: ID='a' (pos 2)

3: EOF (pos 3)

最左推导:

1. E

2. T E'

3. F T' E'

从批量测试的展示结果可以看出，系统能够对文本文件中的多条表达式进行连续解析并生成结构化输出报告：程序能够自动跳过空行，对每条用例依次输出“原始表达式—合法性判定—错误信息（如有）—Tokens—最左推导—语法树”的完整内容，并通过分隔线清晰区分不同用例。

当出现词法或语法错误时，系统不会中断整体流程，而是将该用例的错误定位信息写入报告并继续处理后续输入，从而保证批量测试的鲁棒性与可回溯性。最终生成的 output.txt 既便于教师验收，也便于学生对照定位问题与回归验证，体现了批量测试在效率与一致性方面的优势。

六、总结

本课程设计围绕“递归下降 LL(1) 语法分析”构建了一个可用于教学演示与作业验收的可视化工具，实现了从词法分析到语法分析、从最左推导到语法树展示的完整流程。通过 QTimer 节流实现良好的实时交互体验，通过结构化异常实现精准定位与友好提示，通过批量处理输出报告满足验收需求。系统结构清晰、输出直观，能够有效帮助学习者把编译原理中的抽象过程转化为可观察、可验证、可调试的实际运行过程。

后续可扩展方向包括：加入 FIRST/FOLLOW 表自动计算与展示、导出图形化语法树（如 Graphviz）、支持一元运算或更丰富的文法、支持语义动作（如表达式求值）等，以进一步提升教学互动性与实用性。

七、心得体会

通过本次“递归下降 LL(1) 语法分析”课程设计，我对编译原理中“自顶向下分析”从抽象概念到可运行系统的全过程有了更直观、更系统的理解。以往在课堂上学习 FIRST/FOLLOW、预测分析表、最左推导等内容时，更多是停留在纸面推导与公式记忆上；而当我将文法真正落到递归下降函数、将 lookahead 决策落到代码分支、将推导过程落到可视化输出后，才深刻体会到“文法形式”和“程序结构”之间是一一对应且相互制约的。

本次设计也让我意识到自身仍有提升空间。

例如当前语法树以文本形式展示，虽然便于集成，但在复杂表达式下可读性仍有限；同时 FIRST/FOLLOW 的展示与预测分析表仍停留在隐式逻辑中，若能自动计算并可视化，会更利于教学。后续如果继续完善，我希望能加入 FIRST/FOLLOW 计算模块、支持图形化语法树导出（如 Graphviz），甚至扩展语义动作（如表达式求值），让系统从“语法正确”进一步走向“语义可解释”。

总体而言，这次课程设计不仅巩固了我对 LL(1) 语法分析理论的理解，也训练了我把理论转化为可交付程序的能力。更重要的是，我学会了从“实现算法”进一步思考到“如何让过程可视化、让错误可定位、让工具可使用”，这对后续学习更复杂的编译技术与软件工程开发都有很大帮助。

附录

完整代码如下：

```
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import List, Optional

class LexerError(Exception):
    pass

class ParserError(Exception):
    def __init__(self, message: str, position: int, found: str, expected: List[str]):
        super().__init__(message)
        self.position = position
        self.found = found
        self.expected = expected

@dataclass
class Token:
    type: str
    value: str
    position: int

@dataclass
class Node:
    label: str
    value: Optional[str] = None
    children: Optional[List["Node"]] = None

    def __post_init__(self):
```

```
            if self.children is None:
                self.children = []


def tokenize(text: str) -> List[Token]:
    tokens: List[Token] = []
    i = 0
    length = len(text)
    while i < length:
        ch = text[i]
        if ch.isspace():
            i += 1
            continue
        if ch.isdigit():
            start = i
            while i < length and text[i].isdigit():
                i += 1
            tokens.append(Token("NUM", text[start:i], start))
            continue
        if ch.isalpha() or ch == "_":
            start = i
            i += 1
            while i < length and (text[i].isalnum() or text[i] == "_"):
                i += 1
            tokens.append(Token("ID", text[start:i], start))
            continue
        single_char_tokens = {
            "+": "PLUS",
            "-": "MINUS",
            "*": "MUL",
            "/": "DIV",
            "(": "LPAREN",
```

```
            ")": "RPAREN",
        }
        if ch in single_char_tokens:
            tokens.append(Token(single_char_tokens[ch], ch, i))
            i += 1
            continue
        raise LexerError(f"无法识别的字符 '{ch}' (位置 {i})")
    tokens.append(Token("EOF", "", length))
    return tokens


class Parser:
    NONTERMINALS = {"E", "E'", "T", "T'", "F"}

    def __init__(self, tokens: List[Token]):
        self.tokens = tokens
        self.index = 0
        self.derivation_steps: List[str] = ["E"]
        self.current_sentential_form: List[str] = ["E"]

    def current(self) -> Token:
        return self.tokens[self.index]

    def advance(self) -> Token:
        token = self.tokens[self.index]
        self.index += 1
        return token

    def consume(self, token_type: str, expected_desc: Optional[str] = None) -> Token:
        token = self.current()
        if token.type == token_type:
```

```
                return self.advance()
            expected = expected_desc or token_type
            raise ParserError(
                message=f"位置 {token.position} 处需要 {expected}",
                position=token.position,
                found=token.type,
                expected=[expected],
            )

        def record(self, lhs: str, rhs: List[str]):
            try:
                idx = next(i for i, sym in enumerate(self.current_sentential_form) if sym == lhs)
            except StopIteration:
                return
            if rhs:
                self.current_sentential_form = (
                    self.current_sentential_form[:idx] + rhs + self.current_sentential_form[idx
+ 1 :]
                )
            else:
                self.current_sentential_form = self.current_sentential_form[:idx] +
self.current_sentential_form[idx + 1 :]
            step = " ".join(self.current_sentential_form) if self.current_sentential_form else "ε"
            self.derivation_steps.append(step)

        def parse(self) -> Node:
            tree = self.parse_E()
            if self.current().type != "EOF":
                token = self.current()
                raise ParserError(
                    message=f"多余符号 '{token.value}'", position=token.position,
```

```
found=token.type, expected=["EOF"]
                    )
            return tree

        def parse_E(self) -> Node:
            self.record("E", ["T", "E'"])
            left = self.parse_T()
            right = self.parse_Ep()
            return Node("E", children=[left, right])

        def parse_Ep(self) -> Node:
            token = self.current()
            if token.type == "PLUS":
                self.record("E'", ["+", "T", "E'"])
                op = self.consume("PLUS")
                t_node = self.parse_T()
                ep_node = self.parse_Ep()
                return Node("E'", children=[Node("op", value=op.value), t_node, ep_node])
            if token.type == "MINUS":
                self.record("E'", ["-", "T", "E'"])
                op = self.consume("MINUS")
                t_node = self.parse_T()
                ep_node = self.parse_Ep()
                return Node("E'", children=[Node("op", value=op.value), t_node, ep_node])
            if token.type in {"RPAREN", "EOF"}:
                self.record("E'", [])
                return Node("E'", value="ε")
            raise ParserError(
                message=f"位置 {token.position} 处无法归约 E'",
                position=token.position,
                found=token.type,
```

```
            expected=["+", "-", ")", "EOF"],
        )

    def parse_T(self) -> Node:
        self.record("T", ["F", "T'"])
        f_node = self.parse_F()
        tp_node = self.parse_Tp()
        return Node("T", children=[f_node, tp_node])

    def parse_Tp(self) -> Node:
        token = self.current()
        if token.type == "MUL":
            self.record("T'", ["*", "F", "T'"])
            op = self.consume("MUL")
            f_node = self.parse_F()
            tp_node = self.parse_Tp()
            return Node("T'", children=[Node("op", value=op.value), f_node, tp_node])
        if token.type == "DIV":
            self.record("T'", ["/", "F", "T'"])
            op = self.consume("DIV")
            f_node = self.parse_F()
            tp_node = self.parse_Tp()
            return Node("T'", children=[Node("op", value=op.value), f_node, tp_node])
        if token.type in {"PLUS", "MINUS", "RPAREN", "EOF"}:
            self.record("T'", [])
            return Node("T'", value="ε")
        raise ParserError(
            message=f"位置 {token.position} 处无法归约 T'",
            position=token.position,
            found=token.type,
            expected=["*", "/", "+", "-", ")", "EOF"],
```

```
            )

        def parse_F(self) -> Node:
            token = self.current()
            if token.type == "LPAREN":
                self.record("F", ["(", "E", ")"])
                self.consume("LPAREN", "(")
                e_node = self.parse_E()
                self.consume("RPAREN", ")")
                return Node("F", children=[Node("(", value="("), e_node, Node(")", value=")")])
            if token.type == "ID":
                self.record("F", ["id"])
                tok = self.consume("ID", "标识符")
                return Node("F", children=[Node("id", value=tok.value)])
            if token.type == "NUM":
                self.record("F", ["num"])
                tok = self.consume("NUM", "数字")
                return Node("F", children=[Node("num", value=tok.value)])
            raise ParserError(
                message=f"位置 {token.position} 处无法归约 F",
                position=token.position,
                found=token.type,
                expected=["(", "id", "num"],
            )


  def format_tokens(tokens: List[Token]) -> str:
        lines = []
        for t in tokens:
            if t.type == "EOF":
```

```
                lines.append(f"{len(lines)}: EOF (pos {t.position})")
            else:
                lines.append(f"{len(lines)}: {t.type}='{t.value}' (pos {t.position})")
        return "\n".join(lines)


def format_tree(node: Node) -> str:
        lines: List[str] = []

        def walk(current: Node, prefix: str = "", is_last: bool = True, is_root: bool = True):
            label = current.label if current.value is None else f"{current.label}: {current.value}"
            if is_root:
                lines.append(label)
            else:
                connector = " └── " if is_last else " ├── "
                lines.append(prefix + connector + label)
            new_prefix = prefix + ("     " if is_last else " │   ")
            for idx, child in enumerate(current.children):
                walk(child, new_prefix, idx == len(current.children) - 1, False)

        walk(node)
        return "\n".join(lines)


def format_derivation(steps: List[str]) -> str:
        return "\n".join(f"{idx}. {step}" for idx, step in enumerate(steps, start=1))

ICON_PATH = Path(__file__).with_name("logo.ico")


def load_app_icon() -> QIcon:
```

```
    if ICON_PATH.exists():
        return QIcon(str(ICON_PATH))
    return QIcon()

class ParserWindow(QMainWindow):
    def __init__(self, app_icon: QIcon):
        super().__init__()
        self.setWindowTitle("燕山大学编译原理课程设计")
        self.setWindowIcon(app_icon)
        self.resize(1080, 830)
        self.setMinimumSize(880, 600)
        self.delay_timer = QTimer(self)
        self.delay_timer.setSingleShot(True)
        self.delay_timer.timeout.connect(self.run_analysis)
        self._build_ui()

    def _build_ui(self):
        self.setStyleSheet(
            """
            QMainWindow { background-color: #eef1f6; }
            QLabel#TitleLabel { font-size: 26px; font-weight: 600; color: #122546;
letter-spacing: 1px; }
            QLabel#SubtitleLabel { color: #556079; font-size: 14px; }
            QFrame#card {
                background: rgba(255, 255, 255, 0.92);
                border-radius: 16px;
                border: 1px solid #dce5f7;
            }
            QLabel#cardTitle { font-weight: 600; color: #14213d; }
            QLineEdit {
                border: 1px solid #b8c7e4;
```

```
            border-radius: 12px;
            padding: 10px 14px;
            font-size: 15px;
            background: rgba(255,255,255,0.95);
        }
        QLineEdit:focus { border-color: #4876d3; }
        QPushButton#GhostButton {
            border: 1px solid #c7d3ea;
            border-radius: 10px;
            padding: 6px 18px;
            background: rgba(255,255,255,0.9);
            color: #465169;
        }
        QPushButton#GhostButton:hover { border-color: #4876d3; color: #122546; }
        QPlainTextEdit {
            background: rgba(249, 250, 255, 0.9);
            border: 1px solid #dfe4f3;
            border-radius: 12px;
            font-family: "Fira Code", "Consolas", monospace;
            font-size: 13px;
            padding: 8px;
        }
        QLabel#StatusLabel {
            font-size: 18px;
            font-weight: 600;
        }
        """
    )

    central = QWidget()
    central.setAttribute(Qt.WA_StyledBackground, True)
```

```
self.setCentralWidget(central)
layout = QVBoxLayout()
layout.setContentsMargins(28, 20, 28, 16)
layout.setSpacing(14)
central.setLayout(layout)

title = QLabel("递归下降 LL(1)语法分析")
title.setObjectName("TitleLabel")
subtitle = QLabel("实时递归下降推导 · Tokens / 判定 / 推导 · 语法树")
subtitle.setObjectName("SubtitleLabel")
layout.addWidget(title)
layout.addWidget(subtitle)

input_frame = QFrame()
input_frame.setObjectName("card")
input_frame.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
input_layout = QVBoxLayout(input_frame)
input_layout.setContentsMargins(20, 18, 20, 16)
input_layout.setSpacing(10)
input_layout.addWidget(QLabel("输入表达式，系统将自动实时分析："))

input_row = QHBoxLayout()
input_row.setSpacing(12)
self.input_edit = QLineEdit()
self.input_edit.setPlaceholderText("例如：a+3*(b-2)")
input_row.addWidget(self.input_edit)
button_stack = QVBoxLayout()
button_stack.setSpacing(8)
self.batch_button = QPushButton("批量处理")
self.batch_button.setObjectName("GhostButton")
self.clear_button = QPushButton("清空")
```

```
        self.clear_button.setObjectName("GhostButton")
        button_stack.addWidget(self.batch_button)
        button_stack.addWidget(self.clear_button)
        button_stack.addStretch()
        input_row.addLayout(button_stack)
        input_layout.addLayout(input_row)
        layout.addWidget(input_frame)

        status_frame = QFrame()
        status_frame.setObjectName("card")
        status_frame.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
        status_layout = QVBoxLayout(status_frame)
        status_layout.setContentsMargins(20, 12, 20, 12)
        self.result_label = QLabel("判定：等待输入")
        self.result_label.setObjectName("StatusLabel")
        self.status_hint = QLabel("在输入框内键入表达式即可实时看到结果")
        self.status_hint.setStyleSheet("color: #6c7692;")
        status_layout.addWidget(self.result_label)
        status_layout.addWidget(self.status_hint)
        layout.addWidget(status_frame)

        hint_frame, self.hint_box = self._create_card("提示信息", min_height=60,
wrap=True)
        hint_frame.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
        self.hint_box.setFixedHeight(60)
        self.hint_box.setMaximumBlockCount(3)
        self.hint_box.setWordWrapMode(QTextOption.WordWrap)
        layout.addWidget(hint_frame)

        tokens_frame, self.tokens_box = self._create_card("Tokens", min_height=0)
        derivation_frame, self.derivation_box = self._create_card("最左推导",
```

```
min_height=0)
        tree_frame, self.tree_box = self._create_card("语法树", min_height=0)

        left_splitter = QSplitter(Qt.Vertical)
        left_splitter.setChildrenCollapsible(False)
        left_splitter.addWidget(tokens_frame)
        left_splitter.addWidget(derivation_frame)
        left_splitter.setSizes([1, 1])

        outputs_splitter = QSplitter(Qt.Horizontal)
        outputs_splitter.setChildrenCollapsible(False)
        outputs_splitter.addWidget(left_splitter)
        outputs_splitter.addWidget(tree_frame)
        outputs_splitter.setSizes([1, 1])
        layout.addWidget(outputs_splitter, 1)

        footer = QLabel("计科 23 级 5 班李济岑作品")
        footer.setAlignment(Qt.AlignCenter)
        footer.setStyleSheet("color: #6c7692; font-size: 13px;")
        footer.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Fixed)
        layout.addWidget(footer)

        self.batch_button.clicked.connect(self.run_batch_processing)
        self.clear_button.clicked.connect(self.clear_all)
        self.input_edit.textChanged.connect(self.schedule_analysis)

    def _create_card(self, title: str, min_height: int = 140, wrap: bool = False):
        frame = QFrame()
        frame.setObjectName("card")
        frame.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
        frame_layout = QVBoxLayout(frame)
```

```
        frame_layout.setContentsMargins(20, 16, 20, 20)
        frame_layout.setSpacing(10)
        label = QLabel(title)
        label.setObjectName("cardTitle")
        box = QPlainTextEdit()
        box.setReadOnly(True)
        box.setMinimumHeight(min_height)
        box.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
        box.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)
        box.setHorizontalScrollBarPolicy(Qt.ScrollBarAsNeeded)
        if wrap:
            box.setLineWrapMode(QPlainTextEdit.WidgetWidth)
        else:
            box.setLineWrapMode(QPlainTextEdit.NoWrap)
        frame_layout.addWidget(label)
        frame_layout.addWidget(box, 1)
        return frame, box

    def schedule_analysis(self):
        text = self.input_edit.text()
        if not text.strip():
            self.delay_timer.stop()
            self._show_status("判定：等待输入", mode="neutral", hint="输入框为空，等待内容……")
            self._clear_outputs()
            self.hint_box.clear()
            return
        self._show_status("判定：实时分析中……", mode="neutral", hint="输入暂停 0.25 秒后自动更新结果")
        self.delay_timer.start(250)
```

```
    def run_analysis(self):
        expr = self.input_edit.text().strip()
        if not expr:
            return
        try:
            tokens = tokenize(expr)
        except LexerError as err:
            self._show_status("判定：词法错误", mode="error", hint="请检查是否有非法字符")
            self._clear_outputs()
            self.hint_box.setPlainText(str(err))
            return

        parser = Parser(tokens)
        try:
            tree = parser.parse()
        except ParserError as err:
            expected = ", ".join(err.expected)
            self.tokens_box.setPlainText(format_tokens(tokens))
            self.derivation_box.setPlainText(format_derivation(parser.derivation_steps))
            self.tree_box.setPlainText("(本次失败，未生成语法树)")
            message = (
                f"错误位置：索引 {err.position}，遇到 {err.found}，期望 {expected}\\n提示：{err}"
            )
            self.hint_box.setPlainText(message)
            self._show_status("判定：不合法表达式", mode="error", hint="根据提示修复语法后继续输入")
            return

        self.tokens_box.setPlainText(format_tokens(tokens))
```

```
            self.derivation_box.setPlainText(format_derivation(parser.derivation_steps))
            self.tree_box.setPlainText(format_tree(tree))
            self.hint_box.setPlainText("解析成功，可结合推导与语法树讲解。")
            self._show_status("判定：合法表达式", mode="ok", hint="所有检查通过，语法树已更新")

        def clear_all(self):
            self.delay_timer.stop()
            self.input_edit.clear()
            self._clear_outputs()
            self.hint_box.clear()
            self._show_status("判定：等待输入", mode="neutral", hint="输入框已清空")

        def run_batch_processing(self):
            file_path, _ = QFileDialog.getOpenFileName(
                self, "选择  input.txt", "", "Text Files (*.txt);;All Files (*)"
            )
            if not file_path:
                return
            input_path = Path(file_path)
            try:
                lines = input_path.read_text(encoding="utf-8").splitlines()
            except OSError as err:
                self.hint_box.setPlainText(f"无法读取文件：{err}")
                self._show_status("判定：批量失败", mode="error", hint="请检查文件权限或路径")
                return
            results = []
            case_no = 1
            processed = 0
            for raw in lines:
```

```
                expr = raw.strip()
                if not expr:
                    continue
                processed += 1
                info = self._process_expression(expr)
                block = [f"[用例 {case_no}] 原始：{expr}"]
                block.append(f"结果：{'合法' if info['success'] else '不合法'}")
                if info.get("error"):
                    block.append(info["error"])
                if info.get("tokens"):
                    block.append("Tokens:")
                    block.append(info["tokens"])
                if info.get("derivation"):
                    block.append("最左推导:")
                    block.append(info["derivation"])
                if info.get("tree"):
                    block.append("语法树:")
                    block.append(info["tree"])
                results.append("\n".join(block))
                case_no += 1
            if not results:
                self.hint_box.setPlainText("文件中没有可解析的表达式。")
                self._show_status("判定：批量结束", mode="neutral", hint="空文件或仅含空
行")
                return
            output_path = input_path.with_name("output.txt")
            separator = "\n" + ("-" * 50) + "\n"
            try:
                output_path.write_text(separator.join(results), encoding="utf-8")
            except OSError as err:
                self.hint_box.setPlainText(f"写出 output.txt 失败：{err}")
```

```
            self._show_status("判定：批量失败", mode="error", hint="请检查磁盘权限")
            return
        self.hint_box.setPlainText(
            f"批量处理完成，共 {processed} 条输入。\n 输出文件：{output_path}"
        )
        self._show_status("判定：批量完成", mode="ok", hint="output.txt 已生成")
        QMessageBox.information(self, "批量处理", "已批量完成")

    def _process_expression(self, expr: str):
        info = {"expression": expr, "success": False}
        if not expr:
            info["error"] = "空表达式"
            return info
        try:
            tokens = tokenize(expr)
        except LexerError as err:
            info["error"] = f"词法错误：{err}"
            return info
        parser = Parser(tokens)
        try:
            tree = parser.parse()
        except ParserError as err:
            expected = ", ".join(err.expected)
            detail = (
                f"语法错误：索引 {err.position}，遇到 {err.found}，期望 {expected}"
            )
            info["error"] = detail
            info["tokens"] = format_tokens(tokens)
            info["derivation"] = format_derivation(parser.derivation_steps)
            return info
        info["success"] = True
```

```
            info["tokens"] = format_tokens(tokens)
            info["derivation"] = format_derivation(parser.derivation_steps)
            info["tree"] = format_tree(tree)
            return info

        def _clear_outputs(self):
            self.tokens_box.clear()
            self.derivation_box.clear()
            self.tree_box.clear()

        def _show_status(self, text: str, mode: str, hint: Optional[str] = None):
            colors = {
                "neutral": "#62708f",
                "ok": "#2a9d8f",
                "error": "#c44536",
            }
            self.result_label.setText(text)
            color = colors.get(mode, "#62708f")
            self.result_label.setStyleSheet(f"color: {color}; font-size: 20px; font-weight: 600;")
            if hint is not None:
                self.status_hint.setText(hint)

    def main():
        app = QApplication(sys.argv)
        app.setApplicationName("燕山大学编译原理课程设计")
        icon = load_app_icon()
        if not icon.isNull():
            app.setWindowIcon(icon)
        window = ParserWindow(icon)
        window.show()
        sys.exit(app.exec_())
```

```
if __name__ == "__main__":
    main()
```